

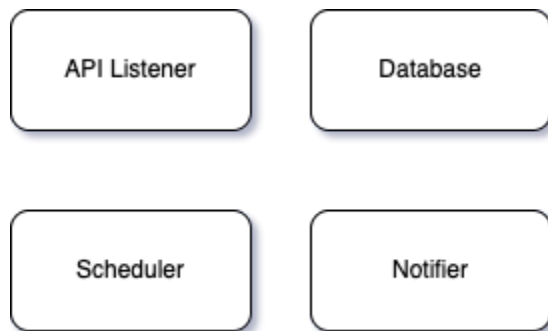
SLA Breach Engine

1.Introduction.

Engineers/Support are in need of a service which can monitor the tickets in Real time, reminding them of the SLA breach that will affect the CSAT. The SLA breaches are defined/tweaked by organisations as per the needs and relevant stakeholders should be reminded of a ticket nearing breach.

This document explains in detail the architectural and design decisions of the SLA Breach Engine. This is an engine which helps anyone who would like to monitor the tickets for SLA Breaches as per their definition.

2. High Level Components



2.1 API Listener

Requirements:

Light weight, fast performing with batch upload support, authentication support, input data validations. This will be executed as a separate service within the container.

		Comments
FasAPI	Fast. Documentation ready(Swagger API), and supports async requests	Requirements

uvicorn	Chosen ASGI framework which works well with FastAPI	Research.
---------	---	-----------

Implementation

		Status
Input Validation	Pydantic with FastAPI. Default Values on timestamps(created/updated, priority, status)	Implemented
Error Handling	Pydantic, Logging.	Implemented.
API Authentication	Simple Secret API key to be sent in API, validated at listener.	Not Implemented. TBD
Documentation	FastAPI has built in documentation with swagger	Implemented.
Async Requests	‘Async’ with FastAPI	Implemented

Example of built in swagger API

localhost:8000/docs#/

SLA Monitor API

0.1.0

OAS 3.0

openapi.json

API for listening to ticket creation

default

POST

/tickets

Create Ticket

GET

/

Read Root

GET

/tickets/{id}

Get Tickets By Id

GET

/dashboard

Get Dashboard

Schemas

HTTPValidationError

detail

items

loc

msg

type

TicketValidator

id

priority

status

created_at

updated_at

customer_tier

ValidationError

loc

msg

type

2.2 Database

Requirement: Provide a database for storing the tickets and SLA alerts. This helps in storage and restart/destruction of services doesn't affect the storage.

Concurrent DB updates.

Should be executed as a separate Service in container.

		Comments
Consistent DB storage. Concurrent writes to DB..	PostgreSQL	Implemented.
Logging/Audit	Structured JSON Logging	Implemented.
Separate Service	Docker compose	Implemented.

2.3 Scheduler

Requirement : A service which looks for the tickets every predefined time interval.